

SCULPTOR Has Your Back(up Path): Carving Interdomain Routes to Services

Paper #559, 12 pages body, 19 pages total

Abstract

Cloud and content providers (Service Providers) serve diverse Internet applications — enterprise services, gaming, video, real-time AI inference — with goals that pull in different directions: low latency for some flows, high availability for others, low cost for bulk transfers, and graceful behavior when peering links fail or flash crowds arrive. To meet these goals Service Providers either We take a different approach with SCULPTOR, a framework that proactively optimizes BGP configurations to satisfy diverse ingress routing objectives-including latency, cost, and capacity constraints-simultaneously. SCULPTOR models Internet routing to approximately solve a large integer optimization problem at scale using gradient descent. We prototyped SCULPTOR on a global public cloud and tested it in real Internet conditions, demonstrating that SCULPTOR handles dynamic loads 28% larger than other solutions using existing Service Provider infrastructure, reduces overloading on links during site failures by up to 40%, and enables Service Providers to route high-priority traffic with a 50% reduction in overloading.

1 Introduction

Cloud/content providers (hereafter Service Providers) enable diverse Internet applications used daily by billions of users. Traditionally, Service Providers used DNS and static BGP advertisements to define a single path from a user to a service, leaving the specific path up to the Internet to determine.

Increasingly, however, these services have diverse requirements that can be challenging to meet with a single path that Service Providers have little control over. For example, enterprise services have tight reliability requirements [1, 2], and new applications such as virtual reality require ≤ 10 ms round trip latency [3] and ≤ 3 ms jitter [4]. Complicating matters, Service Providers must meet these requirements subject to changing conditions such as peering link/site failures [5, 6] and DDoS attacks [7, 8, 9]. Critically, such changes can cause *overload* if the Service Provider cannot handle new traffic

volumes induced by the change. This overload can lead to degraded service for users, hurting reliability [10, 11, 12, 2], and may require manual intervention.

To try to meet different goals and respond to changing conditions, Service Providers adopt solutions that are either (a) too costly, (b) too reactive, or (c) too specific. For example, Service Providers proactively overprovision resources [13, 14, 15], but we demonstrate using traces that bursty traffic patterns can require costly overprovisioning rates as high as 70% despite sufficient global capacity (§2.3). TIPSy responds to changing loads to retain reliability [6], but only does so reactively, which could lead to short-term degraded performance. AnyOpt and PAINTER proactively set up routes to minimize specific objectives such as steady-state latency [16, 1], but it is challenging to extend those approaches to other objectives (§5.4.1). It is also unclear both how to combine these approaches (*e.g.*, retaining low latency under changing traffic loads) and how to apply approaches from one domain to another (*e.g.*, applying egress traffic cost reduction systems [17] to ingress traffic).

We present Service Providers with a flexible framework, SCULPTOR (Scouring Configurations for Utilization-Loss-and Performance-aware Traffic Optimization & Routing), which accepts as input cost, performance, and reliability objectives and outputs BGP advertisements and traffic allocations that help achieve those desired objectives. SCULPTOR is the first system that finds good solutions for ingress interdomain routing objectives such as maximum link utilization, transit cost, and latency for *interdomain* traffic (existing systems work for intradomain traffic, *e.g.*, [18, 19, 20, 21]). SCULPTOR computes BGP advertisements proactively without affecting live traffic, only placing live traffic on new routes after convergence, mitigating risk (§4).

To solve each traffic engineering problem, SCULPTOR efficiently searches over the large BGP advertisement search space ($> 2^{10,000}$ possibilities, *i.e.*, from $> 10,000$ peerings) by modeling how different strategies perform, without having to predict the vast majority of actual paths taken under different configurations since predicting interdomain paths is hard and

measuring them on the Internet is slow (§3.3). SCULPTOR then finds configurations using gradient descent, which is appropriate in our setting due to the horizontal scaling that gradient descent admits (§3.4). This modeling enables SCULPTOR to assess > 1M configurations per measurement on the Internet (§5.4.1), 1,000× more than other solutions. Like other work that uses gradient descent with success (*e.g.*, deep learning), we sacrifice the ability to provide a formal characterization of which objective functions are possible for an approach that lets us approximately optimize multiple criteria (§5)

We prototype and evaluate our framework at Internet scale using the PEERING testbed [22] (§4), which is now deployed at 32 Vultr cloud locations [23]. Vultr is a global public cloud that allows our prototype to issue BGP advertisements via more than 10,000 peerings. To demonstrate SCULPTOR’s utility across different objectives, we separately evaluate our framework on three specific objectives: (a) minimizing latency under unseen traffic conditions, (b) routing latency-sensitive and bulk traffic classes, and (c) directing traffic to lower-cost sites. We compare SCULPTOR’s performance on these problems to that of an unreasonably expensive “optimal” solution (computing the actual optimal is infeasible). We make an anonymized implementation of SCULPTOR available at https://anonymous.4open.science/r/sculptor_stripped-FE7C and will release all code upon acceptance.

For the first objective, we found that, compared to other approaches, SCULPTOR increases the amount of traffic within 10 ms of the optimal by 19.3% in steady-state (meeting that target for 95% of traffic) (§5.3.1), by 11% during link failure, and by 17% during site failure. SCULPTOR also reduces overloading on links during site failures by up to 41% over PAINTER, giving Service Providers more confidence that services will still be available during partial failure (§5.3.2). Moreover, by load balancing traffic on backup paths during peak times, SCULPTOR can satisfy high peak demands with the same infrastructure. SCULPTOR handles flash crowds (*e.g.*, DDoS attacks) at more than 3× expected traffic volume, reducing the amount of overprovisioning that Service Providers need, reducing costs.

For the second objective, SCULPTOR routes bulk low-priority traffic in ways that avoid congesting high-priority traffic, achieving near-optimal latency and reducing congestion by up to 2× (§5.3.4).

Finally, for the third objective, SCULPTOR routes traffic with lower latency and to lower-cost sites than all other solutions (up to 3× closer to optimal) which has implications for power savings, carbon savings, and resilience (§5.3.5).

After decades of intradomain traffic engineering using programmable networking primitives such as virtual output queueing for differentiated service [24] (and others, §7), SCULPTOR brings such control closer to realization in the interdomain setting with a unilaterally deployable framework. Service Providers can use flexible frameworks such as SCULPTOR to help bring us the resilient, performant service that our di-

verse applications increasingly need.

Ethics. This work raises no ethical issues.

2 Motivation

2.1 Variable, Evolving Goals

Evolving Internet use cases are pushing Service Providers to meet diverse requirements. For example, Service Providers increasingly host mission-critical services such as enterprise solutions [1], which are predicted to be a \$60B industry by 2027 [25]. These enterprise solutions require high reliability, whereas low latency is a more important metric for gaming [3], which generates more revenue than the music and movie industries combined [26]. Service Providers are also expanding the set of services they provide—for example, CDNs that traditionally hosted static content are pivoting to offering services like compute [27, 28, 29].

Moreover, as use cases evolve, Service Providers increasingly need to meet performance requirements for *ingress* traffic since that traffic includes, for example, player movements in real-time games, voice and video in enterprise conferences, and video/image uploads for AI processing in the cloud [30, 31]. This reality is a departure from traditional CDN traffic patterns in which ingress traffic was primarily small requests and TCP acknowledgments.

Despite efforts to meet variable service requirements [6, 32, 33, 34, 35, 1, 15], doing so is still challenging due to issues *outside* Service Provider control.

First, Service Providers lack control over which ingress path traffic takes because the source and upstream networks determine the route. BGP computes paths in a distributed fashion where each AS selects the single best path to the destination and propagates it; Service Providers cannot force a source network to select a different path if the source prefers an alternative based on its own local preference. Service Providers cannot even unilaterally control the part of the path closest to them—upgrading peering capacity requires coordination among multiple parties [36].

Second, dynamic factors outside Service Provider control make satisfying requirements even harder. Peering disputes can lead to congestion on interdomain links and inflated paths [5, 37, 38], and DDoS attacks still bring down sites/services [9, 10, 11, 12], despite considerable effort in mitigating attacks [7, 8]. Moreover, recent work [39, 40, 21] and blog posts [41, 42, 43, 44, 45, 46] show that user traffic demands are highly variable due to flash crowds and path changes and so can be hard to plan for.

2.2 Standard Approaches

To meet evolving goals, today most Service Providers either use anycast prefix advertisements to provide low latency and high availability at the expense of some control



Figure 1: In normal operation traffic is split between two sites by directing half the traffic to each prefix using DNS (a). Even though there is enough global capacity to serve all traffic when site B fails, there is no way to split traffic across multiple providers given available paths, leading to overload (b). A proactive solution is to advertise prefix 2 to an additional provider at site A *a priori*, allowing traffic splitting across the two links (c). A simpler but prohibitively expensive solution is to proactively advertise one prefix per peering (d).

[47, 48, 49, 50], or unicast prefix advertisements to direct users to specific sites [51, 52, 36, 53]. Anycast, where Service Providers advertise a single prefix to all peers/providers at all/many sites, offers some natural availability following failures, since BGP automatically reroutes most traffic to avoid the failure after tens of seconds [49]. Prior work shows that this availability comes with higher latency in some cases [54, 48]. Unicast gives clients lower latency than anycast by advertising a unique prefix at each site [51, 53], but can suffer from reliability concerns, taking as much as an hour to shift traffic away from bad routes due to stale DNS records [1].

2.3 New Systems & Limitations

In response to some of the basic limitations of anycast and unicast, others have proposed specialized systems. However, these solutions come with three key limitations.

Too Specific: Some have noted that, by intelligently advertising different prefixes to subsets of peers/providers to offer users more paths, Service Providers can achieve better performance [5, 47, 16, 50, 1]. However, these solutions rely on objective-specific solution methods that do not easily generalize to multi-constraint optimization problems (??).

Figure 1 shows how this specific focus on a single objective could lead to reliability issues, *e.g.*, during a site failure. In normal operation, user traffic is split evenly across two prefixes and achieves low latency (Fig. 1a). However when site B fails, BGP chooses the route through Provider 1 for all traffic, causing link overutilization and subsequent poor performance for all users (Fig. 1b). In Section 5 we show that this overload happens in practice for state-of-the-art systems that provide low (steady-state) latency from users to Service Provider networks such as AnyOpt [16] and PAINTER [1].

Too Reactive: Systems like TIPSy [6] enhance reliability by withdrawing BGP announcements to drain overloaded links. However, changing the set of paths is known to lead to outages [55, 56, 57, 15], and can take time to converge. Ideally, we should modify announcements for live traffic only as a last resort.

Too Costly: Service Providers often overprovision to handle transient peak loads [13, 14, 15], but this approach can incur high costs as Figure 2 demonstrates using link utilization



Figure 2: Planning for peak loads to avoid overloading requires inefficient overprovisioning.

data (per 5 minutes) from OVH cloud [58].

To generate Figure 2 we first split the dataset into successive, non-overlapping 120-day planning periods and compute the 95th percentile ingress link utilization (“near-peak load”) for each link/period. We then assign each link’s capacity for the next period as the near-peak load in the current period multiplied by an overprovisioning factor (X-axis variable). We then compute the average link utilization in the next period and the total number of links on which we see $\geq 100\%$ utilization in at least one 5-minute interval.

Figure 2 plots the median utilization across links and periods and the number of congestive events as we vary the overprovisioning factor. Figure 2 shows that overprovisioning to accommodate peak loads introduces a tradeoff between inefficient utilization and overloading. Low overprovisioning factors between 10% and 30% lead to more efficient utilization (60%-70%) but lead to thousands of congestive events. High overprovisioning factors lead to far less overloading, but only 50% utilization. These events span 41 OVH sites and 1,650 links, indicating a widespread challenge. Results were consistent with larger planning periods, while smaller periods increased congestion.

An important note on cost.: We model deployment cost as correlated with the peering capacity over all links/sites, even though the actual relationship may be complex and additional peering capacity itself is not prohibitively expensive. Upgrading capacity has fixed costs (*e.g.*, fiber, router backplane bandwidth, line card upgrades, CDN servers near peering routers) and variable costs (*e.g.*, power, transit), both of which scale with demand [59, 14, 21, 17]. Hence the high “Link Overprovisioning Factor” to ensure low congestion should be extrapolated out to the general “cost” of upgrading Service Provider serving infrastructure, rather than just a peering link.

Efficient, Proactive Planning is Possible: Despite these limitations, Figure 1c shows that by advertising prefix 2 to provider 2 at site A *a priori*, the Service Provider can split

traffic between the two links during failure, avoiding overloading without (a) reacting to the failure and (b) upgrading deployment capacity.

2.4 Key Challenges

Since a Service Provider has lots of global capacity, dynamically placing traffic on paths to satisfy performance objectives subject to capacity constraints would therefore be simple if all the paths to the Service Provider were always available to all users as in Figure 1d. However, making paths available uses IPv4 prefixes, which are expensive and contribute to the global routing table’s memory footprint. The solution in Figure 1d uses 50% more prefixes than the solution in Figure 1c and, at \$20k per prefix for 10,000 peerings would cost \$200M [60]. IPv6 is not a good alternative as it is not supported by 60% of users according to APNIC [61]. Moreover, adoption is slowing, with adoption in the US stagnating at 50% for a few years, and full adoption possibly taking until 2045 according to APNIC [62] and Google [63]. Prior work noted the same but found that advertising around 50 prefixes was acceptable [16, 1], and most Service Providers advertise fewer than 50 today according to RIPE RIS BGP data [64]. We also verified with engineers at six Service Providers that this challenge is important to address.

Since we cannot expose all the paths by advertising a unique prefix to each connected network, we must find some subset of paths to expose. Finding that right subset of paths to expose that satisfies performance objectives, however, is hard since there are exponentially many subsets to consider, and each subset currently needs to be tested (*i.e.*, advertised via BGP) to see how it performs. The number of subsets is on the order of 2^n to the number of ingresses Service Providers have, which, for some Service Providers (including Vultr, which we measure) [65], is $> 2^{10,000}$. As measuring this many advertisements is intractable, we have to predict how different subsets of paths perform, which is challenging since interdomain routing is difficult to model [66, 6].

3 Methodology

3.1 Overview

SCULPTOR takes as input a Service Provider’s objective, its prefix budget, and, for each user group, its traffic volume and its latency via each peering link it can reach (§3.2.2). It outputs an advertisement configuration that gives users good interdomain paths relative to that objective, and an allocation of traffic to those paths. SCULPTOR computes BGP advertisements proactively, only placing live traffic on them after computation (§4). To search for good advertisements and traffic allocations, SCULPTOR solves two key challenges. First, the only way to know exactly how users will route to a configuration is to advertise it, which can be done at most a

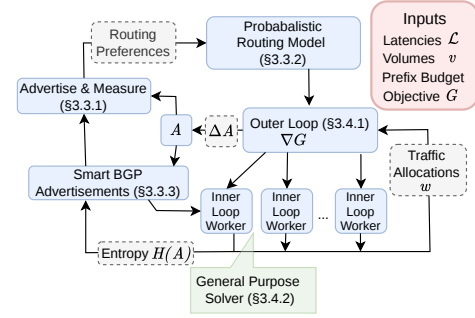


Figure 3: SCULPTOR advertisement computation overview.

few times per hour and so cannot be done for the millions of configurations that an optimization considers. SCULPTOR instead maintains a probabilistic model of how users route, uses it to estimate the objective, and refines it with occasional measurements (§3.3). Every objective value SCULPTOR computes during its search is therefore an estimate, and a noisy one until the model has been refined. Second, the configuration space is large and discrete. SCULPTOR searches it using gradient descent over a relaxed, real-valued configuration which helps smooth the convergence process under a noisy objective (§3.4). Each gradient entry is computed by solving a small traffic-allocation problem (§3.5). Figure 3 shows how these pieces fit together and in Section 5.4.1 we demonstrate that each piece is important for SCULPTOR’s success.

3.2 Problem Formulation

3.2.1 Setting and Routing Model

Service Providers offer their services from tens to hundreds of geo-distributed sites [67]. The sites for a particular service can serve any user, but users benefit from reaching a low-latency site for performance. Sites consist of sets of servers that have an aggregate capacity. Service Providers also connect to other networks at sites via dedicated links or shared IXP fabrics which we call peering links. Each such link also has a capacity. When utilization of a site or link nears/exceeds the capacity, performance suffers, so Service Providers strive to avoid high utilization [5, 21]. Resources can also fail due to, for example, physical failure and misconfiguration.

Users route to the deployment through the public Internet to a prefix over one of the peering links via which that prefix is advertised. The path to a prefix (and therefore peering link) is chosen via BGP. We model interdomain paths from users to the Service Provider as non-overlapping, except at the peering link through which each path ingresses to the Service Providers’ deployment. We identify a path by its ingress link rather than its neighbor AS, because one AS may peer at many sites.

We assume the peering link is the bottleneck of each path, for two reasons. First, this link increasingly represents the most important part of the path for Service Providers due to

Internet flattening [68]. Second, other parts of the path are less interesting for traffic engineering—last-mile bottlenecks are common to all paths for a user, and existing systems perform traffic engineering in intermediate networks. Handling unexpected bottlenecks (*i.e.*, in a peering network) can also be viewed as a path failure, which we evaluate in Section 5.3.2, demonstrating that *SCULPTOR* handles violations of this assumption.

We assume that *SCULPTOR* has a technology for directing traffic towards prefixes. Examples include DNS [51, 53, 47, 52], multipath transport [69, 70, 71], or control-points at/near user networks [72, 1]. DNS offers slow redirection due to caching [1] but is the most deployable, whereas Service Provider-controlled appliances offer precise control but may be hard to deploy. Multipath transport may see wide enough deployment to be used by all Service Providers. Today, MPTCP is installed in iOS [73] and Ubuntu 22 [74]; all applications can use MPQUIC [75]. PAINTER notes that Service Providers with more incentives can invest in more effective direction technology (*e.g.*, deploying MPQUIC) [1].

3.2.2 Inputs

The high-level input is the Service Provider’s goal which they express via an objective function, G . The Service Provider hopes to accomplish this goal via a prefix budget which is generally much less than the number of total ingresses. Service Providers also specify a training time or, equivalently, a maximum number of BGP measurements, since *SCULPTOR* is bottlenecked by BGP measurement time (§3.3).

Oftentimes, objectives are functions of how “well” users route to the deployment, so *SCULPTOR* also accepts information about users. We aggregate users into user groups (UGs), for example /24s, metro-AS pairs, or ASes. The granularity is chosen to be one at which all users in a UG route to the Service Provider the same way, so that a UG has one route per prefix (§3.2.3). We use /24s (§4).

UGs generate steady-state traffic volumes, $v(\text{UG})$, and the Service Provider provisions capacity at links/sites to accommodate this load. A system run by the Service Provider measures latency $\mathcal{L}$ and traffic volumes from UGs to Service Provider peering links l with known capacities $c(l)$, which are also known inputs in prior work [76, 53, 77, 78, 16, 1]. Part of these latency measurements includes measuring which peering links are *reachable* (in a policy-compliant routing sense) from each UG. *SCULPTOR* is resilient to changes in latency (§5.3.2) and traffic (§5.3.2) over time.

3.2.3 General Formulation

Finding advertisement configurations that admit good assignments of user traffic to paths is a multivariate optimization problem over both configurations and traffic assignments. Appendix A tabulates variables and expressions.

We represent an advertisement configuration A as a binary vector. Each entry indicates whether we advertise/do not advertise a particular prefix to a particular peer/provider at a particular site (similar to prior work [16, 1]). Implementing this configuration (*i.e.*, advertising prefixes via BGP sessions) results in routes from UGs to the Service Provider. A UG has one route per prefix but a UG may have their traffic fractionally assigned to different prefixes. The assignment of traffic to resulting routes from this configuration is given by the non-negative real-valued vector w , whose entries specify traffic allocation for each UG along each route (equivalently, prefix).

Announcing a configuration will result in some set of routes (although knowing exactly which routes a priori is challenging), and these routes define which ingress link a UG uses for a prefix. We can think of this process as a routing function R that takes an advertisement configuration A and outputs a map from $\langle \text{prefix}, \text{UG} \rangle$ pairs to ingress links. For example, say $R(A) = f_A$, and $f_A(p, \text{UG}) = l$ — this notation means that the output of an advertisement configuration A under routing R is a function f_A that tells us that users UG reach prefix p via link l . It could be that a configuration leads to no route for some UG to some prefix. We define a function e such that $e(R(A)(p, \text{UG})) = 1$ when there is some route for UG to prefix p under configuration A , and 0 otherwise.

Now suppose the overall metric we want to minimize is G which is a function of both configurations and traffic assignments. Examples include traffic cost, average latency, maximum latency, and their combinations (§3.4). The joint minimization over configurations and traffic assignments can then be expressed as the following.

$$\begin{aligned} \min_{A, w} \quad & G(R(A), w) \\ \text{s.t.} \quad & w(p, \text{UG}) \geq 0; \quad A(p, l) \in \{0, 1\} \quad \forall \text{UG}, p, l \\ & \sum_p w(p, \text{UG}) e(R(A)(p, \text{UG})) = v(\text{UG}) \quad \forall \text{UG} \end{aligned} \quad (1)$$

The first constraint requires that traffic assignments be non-negative and that configurations are binary. The second constraint requires that all user traffic $v(\text{UG})$ is assigned, and none is assigned to nonexistent paths. Capacity constraints depend on the choice of G .

Solving Equation (1) is challenging because we need to compute $G(R(A), w)$, but measuring $R(A)$ exactly requires advertising prefixes in the Internet, which can only be done infrequently to avoid route-flap-dampening and allow convergence. The optimization problem requires evaluating $G(R(A), w)$ for millions of different A , which could take a hundred years at a rate of advertising one configuration every 20 minutes.

Moreover, even if R were known, Equation (1) is not a tractable optimization problem since (1) G depends on A only through R , which is the Internet’s routing rather than an algebraic function of A , and (2) the resulting MIP problem is too large for off-the-shelf solvers.

$$\min_A E_R \left[\min_w G(R(A), w) \right] \quad \text{s.t. the constraints of Equation (1)} \quad (2)$$

We describe each of the three components in Equation (2) in the remainder of this section. The expectation over R represents what we know about routing and how we learn more (§3.3). The outer minimization over A is how we move through configuration space (§3.4.2). The inner minimization over w is how we allocate traffic once routes are fixed (§3.5), and it is where a Service Provider’s specific objective G plugs in (§5.1.2).

3.3 Modeling & Measuring Routes

In Equation (2) we compute expectations over a random process that represents the routing, R , since we cannot exhaustively measure R . (Measuring R takes 20 minutes to 2 hours because of route flap dampening.) Specifically, SCULPTOR models UG paths probabilistically and improves its model over time through relatively few measurements.

3.3.1 Routing Model

We model the routing function and therefore our objective functions probabilistically and update our probabilistic model over time as we measure how UGs route to the deployment (§3.3.2).

A priori, for a given UG towards a given prefix, SCULPTOR treats all reachable ingress link options to which that prefix is advertised as equally likely. One could incorporate prior information about ingress preferences (from economic/routing models, for example), but such complexity is unnecessary for good performance.

We instead learn preferences over time via BGP measurements and observing which ingress the UG takes (§3.3.2). Upon learning that one ingress is preferred over the others receiving that prefix advertisement, we exclude that less-preferred ingress as an option for that UG in all future calculations for all prefixes for which both ingresses are an option. Prior work used a similar routing model [16, 66, 1] but did not extend it to deal with general objectives nor did it treat routing probabilistically.

A useful property of many choices of G is that we do not need to perfectly predict all the routes to solve Equation (1). For example, most UGs have similar latency via many paths to a single site [79], so, when minimizing average latency, we effectively need to predict the *site* each user ingresses at. As we learn which paths UGs prefer the most, our estimate of G on unmeasured configurations concentrates on the true value. An example of this process is shown in Figure 4—SCULPTOR refines a latency estimate towards an unmeasured prefix (blue) using measurements towards other prefixes (red). Even though SCULPTOR has 50% confidence in which path

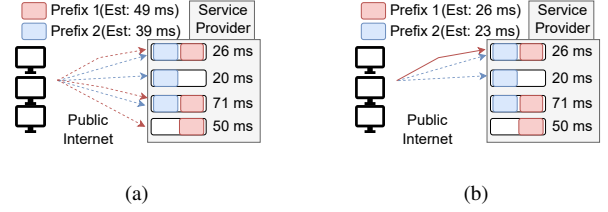


Figure 4: SCULPTOR initially estimates latency from this UG to both the red and blue prefixes with the average over possible ingress latencies (4a). A priori, no prefix is advertised. SCULPTOR then advertises the red prefix and learns that the first ingress is preferred over the third and fourth (4b). SCULPTOR uses this information to refine its latency estimate towards the blue prefix (the third ingress is not a preferred option) without advertising the blue prefix, saving time.

the user takes (*i.e.*, in $R(A)$), SCULPTOR knows that the user’s latency towards the blue prefix will be about 23 ms (*i.e.*, either 26 ms or 20 ms) and so SCULPTOR obtains a better estimate of the objective.

3.3.2 Refining the Model

Probabilistic estimates of G could be inaccurate until we learn UG preferences. To refine its model, SCULPTOR periodically measures $R(A)$ on its current configuration (§3.4) by advertising prefixes and measuring UG preferences by observing which routes users take. To use its BGP measurement budget wisely, SCULPTOR makes two design decisions: \ **What to measure:** SCULPTOR measures the current configuration (as opposed to random or slightly different ones) since it uses a descent-based optimization (§3.4). This type of optimization compares the current advertisement to many adjacent/different ones, so knowing where we are is much more important than knowing where we might be. \ **When to measure:** SCULPTOR measures either when (a) its uncertainty (according to Monte Carlo simulations) exceeds a certain threshold or (b) SCULPTOR has not executed a measurement in many learning iterations. There are many possible choices of “uncertainty”, we choose entropy [?]. Intelligently choosing when to measure offers a slight performance improvement over a fixed measurement schedule (§5.4.1).

3.4 Searching Over Configurations

Equation (2) separates the search over A (this section) from the allocation of traffic, w , (§3.5). We search for A using gradient descent.

3.4.1 Initializing A

We initialize our configuration to be anycast on one prefix and unicast on remaining prefixes (*i.e.*, one prefix per site). If we have more prefixes in our budget, we randomly do/do not advertise those prefixes via random ingresses. We do not

use a completely random initialization since `anycast` and `unicast` have their benefits (§2.2).

3.4.2 Gradient Descent

A reasonable choice to solve an optimization problem with too many binary decision variables for an off-the-shelf solver is a descent-based algorithm (*i.e.*, comparing adjacent options to the current best guess and picking an adjacent option as the next best guess). An additional challenge in our setting is that we only have access to noisy estimates of the objective because we cannot measure the objective at adjacent configurations without exhausting measurement budget.

SCULPTOR uses gradient descent as a powerful solution to this problem since gradient descent naturally incorporates *memory* about how useful advertisements are rather than committing to any single comparison, which is what greedy and random searches used by prior systems do [16, 1]. Gradient descent here is akin to SGD in Deep Learning since we Monte Carlo simulate (*i.e.*, subsample) the true distribution of objectives and move in the direction of the gradient estimate.

To apply gradient descent to our problem, we create a real-valued auxiliary variable to A , $\tilde{A}$, with entries between 0 and 1 representing “confidence” values that an advertisement to the corresponding ingress will be beneficial. For coordinate (p, l) , SCULPTOR evaluates the inner objective (§3.5) with $A(p, l)$ set to 1 and to 0, holding the rest of the thresholded configuration fixed. The difference, scaled by the derivative of a sigmoid at $\tilde{A}$, is the gradient entry (similar to prior work [80]). Since there are too many gradients to compute, we subsample gradient entries and track the largest ones (as in prior work [81]). We then threshold $\tilde{A}$ at 0.5 to translate real-valued confidences to whether or not we should advertise a prefix at an ingress.

We implement SCULPTOR with Nesterov’s Accelerated Gradient Descent (§3.4.2) in Python [86]. We use RMSProp with an initial learning rate of 1.0.

Scaling: Minimization in the outer loop scales with the product of the number of ingresses and number of prefixes but is completely parallelizable. Hence, the bottleneck is effectively the prefix/path measurement period (at least 20 minutes per measurement).

Convergence: Gradient descent converges to a local minimum for bounded objectives [82]. Our evaluations show that SCULPTOR finds good solutions over a wide range of emulated topologies (§4.1) and converges quickly with thousands of UGs and $\langle \text{peering}, \text{prefix} \rangle$ pairs (§5). Allocating higher prefix budgets and adding richer advertisement capabilities (*e.g.*, BGP community tagging) can lead to convergence to a better minimum, which is an area for future work.

3.4.3 Termination

SCULPTOR stops gradient descent either when the BGP measurement (time) budget has been exhausted or the average change in objective over iterations falls below a small threshold. We stop training when measurement budget has been exhausted since SCULPTOR may make poor decisions if it has a bad estimate of the routing function. For example, with a waiting time between measurements of an hour and a maximum training time of 2 days, SCULPTOR terminates within 48 measurements.

3.5 Allocating Traffic

This section specifies the inner minimization over traffic allocations in Equation (2). Given our routing model (§3.3.1) and current advertisement (§3.4), we Monte Carlo simulate user path choices. For each set of path choices, we solve for the optimal value of G using a general-purpose solver. The expectation in Equation (2) then returns the average over Monte Carlo samples as *the* objective estimate for use in gradient descent.

We solve traffic allocations (§3.5) with SciPy HIGHs **TBD: scipy-highs**.

Scaling: This component scales with the number of Monte Carlo samples and the scaling behavior of each traffic allocation, which depends on the objective, G .

Convergence: depends on the objective, G . SCULPTOR accepts any G for which, given fixed routes, the allocation problem is efficiently solvable. All objectives that we evaluate in Section 5.1.2 are linear programs.

4 Evaluation Setup

We prototype SCULPTOR on the PEERING testbed [22], which is now available at 32 Vultr cloud sites. We describe how we built SCULPTOR on the real Internet and how we *emulate* a Service Provider including their clients, traffic volumes, and resource capacities. (We are not a Service Provider and so could not obtain actual volumes/capacities, but our extensive evaluations (§5) demonstrate SCULPTOR’s potential in an actual Service Provider and our open/reproducible methodology provides value to the community.)

In practice, Service Providers can use SCULPTOR *safely* by only shifting production traffic onto prefixes after SCULPTOR’s announcement exploration has converged and by integrating SCULPTOR into automated route configuration frameworks. More details about how a Service Provider may do so are in Appendix B.

4.1 Emulating Client Traffic

We measure actual client latency from IP addresses to our PEERING prototype as in prior work [1], selecting targets

according to assumptions about Vultr cloud’s client base.

We first tabulate a list of 5M IPv4 targets that respond to ping via probing each /24. Vultr informs cloud customers of which prefixes are reachable via which peers, and we use this information to tabulate a list of peers and clients reachable through those peers. We then measure latency from all clients to each peer individually by advertising a prefix solely to that peer using Vultr’s BGP action communities and pinging clients from Vultr. We also measure performance from all clients to all providers individually, as providers provide global reachability.

In our evaluations, we limit our focus to clients who had a route through at least one of Vultr’s direct peers (we exclude route server peers [83]). Vultr likely peers with networks with which it exchanges a significant amount of traffic [36], so clients with routes through those peers are more likely to be “important” to Vultr. We found 700k /24s with routes through 1086 of Vultr’s direct ingresses. We removed clients whose lowest latency to Vultr was 1 ms or less, as these were assumed to be addresses related to infrastructure, leaving us with measurements from 666k /24s to 825 Vultr ingresses.

We emulate traffic volumes in two ways: First, we randomly choose the total traffic volume of a site as a number between 1 and 10 and then divide volume up randomly among clients that anycast routes to that site. Client volumes in a site are set to be within one order of magnitude of each other. While potentially unrealistic, testing many different random volumes demonstrates SCULPTOR’s robustness to assumptions about a particular distribution. We avoid Zipf-like distributions as they lead to trivial solutions where a small fraction of sites dominate importance.

Second, we derive realistic volumes using APNIC AS populations [84], shown to be representative of Service Provider traffic volumes [85]. We assign each AS its aggregate population and divide it evenly among its clients.

We found that the high-level results shown in Section 5 were the same using both emulated and APNIC volumes, and so report results for emulated traffic.

4.2 Deployments

Experiments in the Internet. We assess how SCULPTOR performs on the Internet using RIPE Atlas probes [87], which represent a subset of all clients. RIPE Atlas allows us to measure paths (and thus ingress links) to prefixes we announce from PEERING, which SCULPTOR needs to refine its model (§3). We limit the scale of our deployment to 10 sites to avoid reaching RIPE Atlas daily probing limits (15k traceroutes/day). We prioritize geographic density over coverage to create a challenging routing surface with subtle unicast differences. These 10 sites include major hubs across North America, Europe, and South America. Choosing RIPE Atlas probes to maximize network coverage and geographic diversity, we select probes from 972 networks in 38 countries

which have paths to 484 unique ingresses. Each probe has paths via approximately 60 ingresses. We use 12 prefixes and change advertisements at most once per hour to avoid route-flap-dampening.

Emulations. We also evaluate SCULPTOR by emulating user paths which allows us to conduct more extensive evaluations, as experiments take less time and use clients in more networks. We compute solutions over many random routing preferences, demands, and subsets of sites to demonstrate that SCULPTOR’s benefits are not limited to a specific deployment. We evaluate SCULPTOR over deployments of size 3, 5, 10, 15, 20, 25, and 32 sites. Each size deployment is run at least 10 times with random subsets of UGs, UG demands, and routing preferences. The distribution of the number of /24s per peer is Pareto-like, so we consider random subsets of /24s through each ingress in a way that balances the number of unique /24s per ingress. Over all scenarios, we consider paths from clients in 52k prefixes (representing 31% of APNIC population [84]) to 873 ingresses. We use one tenth of the number of ingresses as the number of prefixes in our budget (10 prefixes for 3 site deployments, 60 prefixes for 32 site deployments). Prior work found that using tens of prefixes to improve performance was a reasonable cost [16, 1].

4.3 Setting Resource Capacities

We assume that resource capacities are overprovisioned proportional to their usual load. However, we do not know the usual load of Vultr links and cannot even determine which peering link that traffic to one of our prefixes arrives on, as Vultr does not give us this information. (This limitation exists since we are not a Service Provider, but a Service Provider could measure this using IPFIX, for example.) We overcome this limitation using two methods corresponding to our two deployments in Section 4.2.

Experiments in the Internet. For our first method of inferring client ingress links, we advertise prefixes into the Internet using the PEERING testbed [22], and measure actual ingress links to those prefixes using traceroutes from RIPE Atlas probes [87]. Specifically, we perform IP to AS mappings and identify the previous AS in the path to Vultr. This approach has limited evaluation coverage, as RIPE Atlas probes are in a few thousand networks. In cases where we cannot infer the ingress link from a traceroute, we use the closest latency from the traceroute to the clients’ (known) possible ingresses. For example, if an uninformative traceroute’s latency was 40 ms to Vultr’s Atlanta site and a client was known to have a 40 ms path through AS1299 at that site, we say the ingress link was AS1299 at Atlanta.

Emulations. The second method we use to infer ingress links is emulating user paths by assuming we know all user routing preferences (§3.2). We use a preference model where clients prefer peers over providers, and clients have a preferred provider. When choosing among multiple ingresses for the